BUILD YOUR FIRST AI AGENT

Curriculum Plan

A hands-on introduction to Agentic AI using Python and LLM APIs

Technical Trainer Assessment

Course duration: 3 days × 3 hours per day | Beginner level

TABLE OF CONTENTS

TABLE OF CONTENTS 2

1. Course Overview 3

2. Course Purpose 3

3. Learning Objectives 3

4. Course Structure at a Glance 4

5. Detailed Three-Day Curriculum 4

Day 1 — Understanding AI Agents (3 hours) 4

- **Topics Covered** 5
- **Hands-on Lab 1 — Build Your First LLM Application** 5

Day 2 — Building Your First AI Agent (3 hours) 6

- **Topics Covered** 6
- **Hands-on Lab 2 — Build a Smart Expense Assistant** 7

Day 3 — Agentic AI Applications and Mini Project (3 hours) 7

- **Topics Covered** 8
- **Hands-on Lab 3 — Build a Student Study Assistant** 8

6. Teaching and Delivery Approach 8

1. Course Overview

Course title: Build Your First AI Agent — From LLM Fundamentals to Tool-Calling Agents.

Course duration: 3 days × 3 hours per day. Teaching level: Beginner.

Target audience: 2nd/3rd-year engineering students and junior developers with basic Python or JavaScript knowledge.

Prerequisites: Variables, functions, loops and conditional statements. A basic understanding of APIs and JSON is helpful but not mandatory. No prior LLM or Agentic AI experience is required.

Technology: Python 3, an LLM API with tool-calling support, JSON, and VS Code or Google Colab.

2. Course Purpose

This mini-course introduces students to AI agents through simple explanations, guided demonstrations and practical Python exercises. Students progress from understanding LLMs and APIs to building a tool-calling agent and a small agent-based application.

Teaching principle: Help students understand how an agent works, build one themselves, and identify where it can fail.

3. Learning Objectives

1. Explain the difference between Artificial Intelligence, Generative AI, LLMs and Agentic AI.
2. Understand how an LLM communicates through an API.
3. Explain the difference between a traditional chatbot and an AI agent.
4. Describe function calling and tool integration.
5. Explain and implement the Plan → Act → Observe agent loop.
6. Build a basic AI agent using Python.
7. Connect tools such as calculators, APIs and search functions.
8. Identify common agent failures, hallucinations and security risks.
9. Build a small agent-based application independently.

4. Course Structure at a Glance

Day	Theme	Main focus	Hands-on lab	Daily deliverable
Day 1	Understanding AI Agents	AI fundamentals, LLMs, APIs and agent architecture	First LLM-powered Q&A application	Working LLM-powered Q&A application
Day 2	Building Your First AI Agent	Tool calling, agent loop, Python implementation, memory and context	Smart Expense Assistant	Functional tool-calling AI agent
Day 3	Agentic AI Applications and Mini Project	Agent workflows, multi-agent concepts, RAG, guardrails and evaluation	Student Study Assistant	Small agent-based student learning application

5. Detailed Three-Day Curriculum

Day 1 — Understanding AI Agents (3 hours)

Learning outcome: Students understand the fundamentals of AI and LLMs and can explain how an AI agent differs from a normal chatbot.

Time	Duration	Topic	Teaching approach
0–15 min	15 min	Introduction to AI	Real-world examples and discussion
15–40 min	25 min	AI, ML, Deep Learning and Generative AI	Concept explanation and examples

40–65 min	25 min	What is an LLM? Tokens, prompts, context and inference	Simple examples and walkthrough
65–75 min	10 min	Break	—
75–105 min	30 min	LLM APIs: requests, responses, JSON and API keys	API request/response demonstration
105–135 min	30 min	Chatbot vs AI Agent	Compare practical examples
135–165 min	30 min	Agent architecture: goal, reasoning, tools and memory	Architecture diagram and discussion
165–180 min	15 min	Hands-on Lab 1	Build a first LLM-powered application

Topics Covered

- Artificial Intelligence vs Machine Learning.
- Deep Learning and neural networks.
- Generative AI and Large Language Models.
- How an LLM generates responses.
- Prompt engineering fundamentals.
- System prompts, user prompts and context.
- What is an AI agent?
- Reactive chatbot vs tool-using agent.
- Introduction to APIs and authentication.

Hands-on Lab 1 — Build Your First LLM Application

Students create a basic Python application that accepts a question, sends it to an LLM API and displays the generated response.

Expected workflow: User enters a question → Python application → LLM API request → LLM generates a response → Display the answer.

Example prompt: “What is Artificial Intelligence?”

Expected deliverable: A working LLM-powered question-answering application.

Day 2 — Building Your First AI Agent (3 hours)

Learning outcome: Students understand how agents make decisions, call tools, observe tool results and produce responses.

Time	Duration	Topic	Teaching approach
0–15 min	15 min	Day 1 revision	Quick quiz
15–40 min	25 min	Agent architecture	Architecture diagram
40–70 min	30 min	Tool calling and function calling	Practical examples
70–80 min	10 min	Break	—
80–110 min	30 min	Plan → Act → Observe	Step-by-step walkthrough
110–145 min	35 min	Build an AI agent	Live coding
145–170 min	25 min	Memory and context	Conversation history example
170–180 min	10 min	Hands-on Lab 2	Tool integration exercise

Topics Covered

- What makes an agent autonomous?
- Agent goals and task decomposition.
- LLM tool/function calling.
- Function definitions and JSON schemas.
- Agent loop: Plan → Act → Observe.
- Tool selection and execution.
- Short-term memory and conversation history.
- Error handling and stopping conditions.

Hands-on Lab 2 — Build a Smart Expense Assistant

Students build an assistant that can add an expense, calculate total expenses, categorize spending and display expense summaries.

Example: Student: “I spent ₹250 on lunch.” The model requests add_expense(amount=250, category='Food'). The Python application executes the function and returns the result. The agent responds: “Successfully added ₹250 under Food.”

The agent should decide when to call add_expense() and when to call get_total_expenses().

Expected deliverable: A functional tool-calling AI agent.

Day 3 — Agentic AI Applications and Mini Project (3 hours)

Learning outcome: Students understand how to design, evaluate, secure and extend agent-based applications.

Time	Duration	Topic	Teaching approach
0–15 min	15 min	Revision	Discussion
15–40 min	25 min	Types of AI agents	Real-world use cases
40–70 min	30 min	Multi-agent systems	Architecture examples
70–80 min	10 min	Break	—
80–105 min	25 min	Agent frameworks: LangChain and LangGraph	Framework overview
105–130 min	25 min	Agent memory and RAG	Practical examples
130–150 min	20 min	Failure modes and guardrails	Case studies
150–170 min	20 min	Mini project	Build and test
170–180 min	10 min	Assessment and recap	Quiz and feedback

Topics Covered

- ReAct-style agent workflows.
- Single-agent vs multi-agent systems.
- Agent memory and context management.
- Retrieval-Augmented Generation (RAG).
- Introduction to LangChain and LangGraph.
- Tool errors and infinite loops.
- Hallucinations and incorrect tool selection.
- Prompt injection and unsafe tool execution.
- Human approval and guardrails.
- Agent evaluation and performance measurement.
- Real-world applications.

Hands-on Lab 3 — Build a Student Study Assistant

Students build a basic assistant that accepts a study question, searches a predefined collection of notes, generates a summary, creates quiz questions and answers relevant questions from the provided material.

Expected deliverable: A small agent-based student learning application.

6. Teaching and Delivery Approach

- Use simple examples before introducing technical terminology.
- Connect each concept to the Smart Expense Assistant or Student Study Assistant.
- Use diagrams to explain API communication, tool calling and the agent loop.
- Ask short questions during the session to check student understanding.
- Give students time to implement and test the functions themselves.
- Discuss both successful and failed tool executions.